ASSIGNMENT 1 - UNSUPERVISED DEEP LEARNING (AIMLZG533)

Submitted by

Group 93

👥 Team Members & Contribution Mapping

#	Name	BITS ID	Contribution Areas
1	**Shahabuddin**	2025AE05328	① Data preprocessing pipeline (grayscale conversion, resize to 28×28, [50, 200] normalization) ② Train / validation / test split
2	**Prajwal Shetty K P**	2025AE05434	① Task 1 — Standard & Randomized PCA ② Logistic-regression classifier & ROC curves ③ Reconstruction SNR
3	*[Member 3 Name]*	*[BITS ID]*	Task 2 — Tied-weight linear autoencoder, PCA-vs-AE subspace comparison
4	*[Member 4 Name]*	*[BITS ID]*	Task 3 — Shallow nonlinear, deep dense & deep convolutional autoencoders
5	*[Member 5 Name]*	*[BITS ID]*	Evaluation & analysis, visualization & reporting

Problem statement

This assignment studies representation learning with several variants of autoencoders. We use two datasets:

1. **CIFAR-10** from Keras, converted to gray-level images.
2. **MNIST** handwritten digits, which are already gray-scale.

Both datasets are rescaled to 28×28, their intensities are normalized to the range [50, 200], and every experiment uses the same random split of 70% training, 20% validation and 10% test data.

The work is organised into three tasks:

- **Task 1** compares standard PCA against randomized PCA at 30 components, uses the 30-dimensional projections to train a 10-class logistic-regression classifier, plots per-class ROC curves on the test set, and reports the average reconstruction SNR.

- **Task 2** trains a single-layer linear autoencoder with tied weights and unit-norm encoder vectors, then measures how close its 30-dimensional subspace is to the PCA subspace, both visually and through principal angles.
- **Task 3** looks at what nonlinearity, depth and convolution add on top of the linear baseline, with the latent dimensionality held fixed at 30.

Notebook organization

Sections 1 and 2 build the shared preprocessing pipeline and the metric / plotting helpers that all three tasks reuse, so the train / validation / test split and the training-set mean are identical throughout. Sections 3 to 5 contain the three tasks, each followed by a short discussion of the results, and Section 6 collects the overall conclusions.

1. Environment and reproducibility

All random number generators (NumPy, TensorFlow, Keras) are seeded with the same value so the split, the PCA fits and the autoencoder training are reproducible from run to run.

In [1]:
```
import io
import time
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler, label_binarize
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, roc_curve, auc
from scipy.optimize import linear_sum_assignment

%matplotlib inline
plt.rcParams["figure.dpi"] = 110
plt.rcParams["figure.facecolor"] = "white"
warnings.filterwarnings("ignore")

SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)
try:
    keras.utils.set_random_seed(SEED)
except Exception:
```

```
    pass

print("TensorFlow", tf.__version__)
print("GPU devices:", tf.config.list_physical_devices("GPU"))
```

```
TensorFlow 2.20.0
GPU devices: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

The experiment-wide constants are kept in one place. `N_COMPONENTS = 30` is the latent size used by PCA and by every autoencoder, so the comparisons in Tasks 2 and 3 are all made at the same representational budget.

In [2]:
```
N_COMPONENTS = 30                # latent / PCA dimensionality used everywhere
IMG_SIZE = 28
INTENSITY_RANGE = (50.0, 200.0)
N_CLASSES = 10
BATCH_SIZE = 256
```

2. Data preparation

The same four steps are applied to both datasets:

1. **Load and make gray-scale.** MNIST is already single-channel. CIFAR-10 is converted with the standard luma weighting (`tf.image.rgb_to_grayscale`) and resized from 32×32 to 28×28 with bilinear interpolation, so both datasets share a 784-pixel input space.
2. **Normalize intensity to [50, 200].** Pixel values are linearly mapped from their original `[min, max]` onto `[50, 200]` . The assignment fixes this band; keeping a non-zero floor of 50 also means the SNR denominator used later is not dominated by near-zero background pixels.
3. **Split 70 / 20 / 10.** A stratified split keeps the class proportions identical across the three sets. We first hold out 30% of the data, then cut that portion into 20% validation and 10% test.
4. **Mean-center with the training mean.** PCA needs centered data, and the same training-set mean vector is subtracted from the validation and test sets. Every reconstruction adds this mean back before any SNR calculation or visualization, so all image-space quantities stay in the original [50, 200] domain.

The function returns both the centered arrays (used for fitting) and the uncentered [50, 200] train / test arrays (used as the SNR reference and for display).

In [3]:
```
def load_and_preprocess(dataset_name):
    """Load a dataset, convert to 28x28 gray, rescale to [50, 200], split
    70/20/10 (stratified) and mean-center with the training mean.

    Returns a dict holding the centered splits used for model fitting, the
    uncentered [50, 200] train/test arrays used as the SNR reference, the
    labels, and the training mean vector.
```

```
    """
    name = dataset_name.lower()
    if name == "mnist":
        (x1, y1), (x2, y2) = keras.datasets.mnist.load_data()
        X = np.concatenate([x1, x2]).astype("float32")[..., None]
        y = np.concatenate([y1, y2]).ravel()
    elif name == "cifar10":
        (x1, y1), (x2, y2) = keras.datasets.cifar10.load_data()
        X = np.concatenate([x1, x2]).astype("float32")
        y = np.concatenate([y1, y2]).ravel()
        X = tf.image.rgb_to_grayscale(X).numpy()
        X = tf.image.resize(X, [IMG_SIZE, IMG_SIZE]).numpy()
    else:
        raise ValueError(name)

    X = X.reshape(len(X), -1)                       # (N, 784)

    lo, hi = INTENSITY_RANGE
    xmin, xmax = float(X.min()), float(X.max())
    X = lo + (X - xmin) / (xmax - xmin) * (hi - lo)

    X_tr, X_tmp, y_tr, y_tmp = train_test_split(
        X, y, test_size=0.30, random_state=SEED, stratify=y)
    X_val, X_te, y_val, y_te = train_test_split(
        X_tmp, y_tmp, test_size=1 / 3, random_state=SEED, stratify=y_tmp)

    mean_vec = X_tr.mean(axis=0)
    return {
        "name": name,
        "X_train": (X_tr - mean_vec).astype("float32"),
        "X_val":   (X_val - mean_vec).astype("float32"),
        "X_test":  (X_te - mean_vec).astype("float32"),
        "X_train_orig": X_tr.astype("float32"),
        "X_test_orig":  X_te.astype("float32"),
        "y_train": y_tr, "y_val": y_val, "y_test": y_te,
        "mean_vec": mean_vec.astype("float32"),
    }
```

In [4]:

```
DATA = {name: load_and_preprocess(name) for name in ["mnist", "cifar10"]}

rows = []
for name, d in DATA.items():
    rows.append({
        "dataset": name,
        "train": d["X_train"].shape[0],
        "val": d["X_val"].shape[0],
        "test": d["X_test"].shape[0],
        "features": d["X_train"].shape[1],
        "intensity min": round(float(d["X_train_orig"].min()), 1),
        "intensity max": round(float(d["X_train_orig"].max()), 1),
    })
pd.DataFrame(rows).set_index("dataset")
```

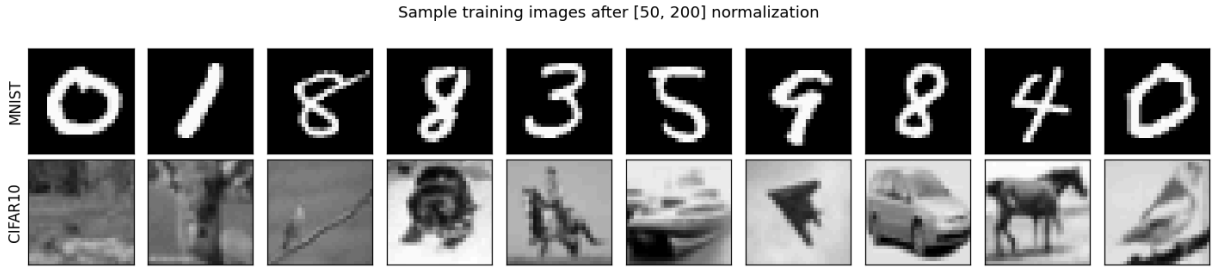
```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 1673s 10us/step
```

Out[4]:

dataset	train	val	test	features	intensity min	intensity max
mnist	49000	14000	7000	784	50.0	200.0
cifar10	42000	12000	6000	784	50.0	200.0

In [5]:

```
fig, axes = plt.subplots(2, 10, figsize=(13, 3))
for r, name in enumerate(["mnist", "cifar10"]):
    d = DATA[name]
    for c in range(10):
        axes[r, c].imshow(d["X_train_orig"][c].reshape(28, 28), cmap="gray",
                          vmin=50, vmax=200)
        axes[r, c].set_xticks([]); axes[r, c].set_yticks([])
    axes[r, 0].set_ylabel(name.upper(), fontsize=11)
fig.suptitle("Sample training images after [50, 200] normalization")
plt.tight_layout(); plt.show()
```



3. Shared metrics and plotting helpers

Two quantities are used repeatedly.

Reconstruction SNR (dB). For each image we compute `10 * log10( ||x||^2 / ||x - x_hat||^2 )` and average over the set. It is evaluated in the [50, 200] domain, that is after adding the training mean back to a reconstruction, so a larger value means a more faithful image.

Principal angles between subspaces. Given two sets of `k` basis vectors, orthonormalize each set and take the singular values of the cross-product matrix. Those singular values are the cosines of the principal angles `theta_1 <= ... <= theta_k`. Angles near zero mean the two subspaces point in the same directions. This is the natural way to compare a PCA basis with an autoencoder weight matrix, because it ignores the order, the sign and any linear mixing of the individual vectors and looks only at the span (Bjorck and Golub, 1973).

In [6]:
```
def average_snr_db(original, reconstructed):
    """Mean per-image SNR in dB, computed in the [50, 200] domain."""
    signal = np.sum(original ** 2, axis=1)
    noise = np.sum((original - reconstructed) ** 2, axis=1)
    noise = np.where(noise == 0.0, 1e-12, noise)
    return float(np.mean(10.0 * np.log10(signal / noise)))


def principal_angles_deg(A, B):
    """Principal angles (degrees, ascending) between the column spaces of A and B."
    Qa, _ = np.linalg.qr(A)
    Qb, _ = np.linalg.qr(B)
    sv = np.linalg.svd(Qa.T @ Qb, compute_uv=False)
    return np.degrees(np.arccos(np.clip(sv, -1.0, 1.0)))


def subspace_report(V_ref, W):
    """How close span(W) is to span(V_ref); both matrices are (D, k)."""
    angles = principal_angles_deg(V_ref, W)
    cos_t = np.cos(np.radians(angles))
    Qa, _ = np.linalg.qr(V_ref)
    Qb, _ = np.linalg.qr(W)
    proj_gap = np.linalg.norm(Qa @ Qa.T - Qb @ Qb.T, "fro") / np.sqrt(2 * V_ref.sha
    return {
        "mean_angle_deg": float(angles.mean()),
        "max_angle_deg": float(angles.max()),
        "mean_cos": float(cos_t.mean()),
        "chordal_distance": float(np.sqrt(np.sum(1.0 - cos_t ** 2))),
        "projection_gap": float(proj_gap),
        "angles": angles,
    }


def align_columns(V_ref, W):
    """Reorder and sign-flip the columns of W so that column i is the one that
    best matches V_ref[:, i], using a maximum-weight matching on |cosine|.
    Only used to make the visual panels readable row by row."""
    C = V_ref.T @ W
    _, col = linear_sum_assignment(-np.abs(C))
    W_al = W[:, col].copy()
    signs = np.sign(np.sum(V_ref * W_al, axis=0))
    signs[signs == 0] = 1.0
    return W_al * signs, col


def logreg_test_accuracy(Z_train, y_train, Z_test, y_test):
    """Standardize -> multinomial logistic regression -> test accuracy."""
    clf = make_pipeline(StandardScaler(),
                        LogisticRegression(max_iter=3000, solver="lbfgs"))
    clf.fit(Z_train, y_train)
    return float(accuracy_score(y_test, clf.predict(Z_test))), clf


def plot_roc_ovr(y_true, y_score, title, ax):
    """One-vs-rest ROC for all 10 classes on one axis; returns macro-average AUC.""
```

```
    y_bin = label_binarize(y_true, classes=list(range(N_CLASSES)))
    aucs = []
    for i in range(N_CLASSES):
        fpr, tpr, _ = roc_curve(y_bin[:, i], y_score[:, i])
        a = auc(fpr, tpr)
        aucs.append(a)
        ax.plot(fpr, tpr, lw=1.3, label=f"class {i}  (AUC {a:.3f})")
    macro = float(np.mean(aucs))
    ax.plot([0, 1], [0, 1], "k--", lw=1)
    ax.set_xlabel("false positive rate")
    ax.set_ylabel("true positive rate")
    ax.set_title(f"{title}\nmacro-average AUC = {macro:.3f}")
    ax.legend(fontsize=7, loc="lower right")
    ax.grid(alpha=0.3)
    return macro, aucs


def show_image_grid(columns, title, n=30, ncol=10, size=28):
    """Show the first n columns of a (D, k) matrix as size x size gray images."""
    n = min(n, columns.shape[1])
    nrow = int(np.ceil(n / ncol))
    fig, axes = plt.subplots(nrow, ncol, figsize=(1.3 * ncol, 1.4 * nrow))
    axes = np.atleast_2d(axes)
    for j in range(nrow * ncol):
        ax = axes.flat[j]
        ax.axis("off")
        if j < n:
            ax.imshow(columns[:, j].reshape(size, size), cmap="gray")
            ax.set_title(str(j + 1), fontsize=7)
    fig.suptitle(title)
    plt.tight_layout(); plt.show()


def show_reconstructions(orig, recon, title, n=10, size=28):
    fig, axes = plt.subplots(2, n, figsize=(1.5 * n, 3.3))
    for i in range(n):
        axes[0, i].imshow(orig[i].reshape(size, size), cmap="gray", vmin=50, vmax=2
        axes[1, i].imshow(recon[i].reshape(size, size), cmap="gray", vmin=50, vmax=
        axes[0, i].axis("off"); axes[1, i].axis("off")
    axes[0, 0].set_title("original", loc="left", fontsize=9)
    axes[1, 0].set_title("reconstruction", loc="left", fontsize=9)
    fig.suptitle(title)
    plt.tight_layout(); plt.show()


def plot_history(histories, title):
    """histories: {label: keras History}. Log-scale MSE vs epoch,
    train solid and validation dashed in the same colour."""
    fig, ax = plt.subplots(figsize=(7, 4.3))
    for label, h in histories.items():
        ep = range(1, len(h.history["loss"]) + 1)
        line, = ax.plot(ep, h.history["loss"], lw=1.7, label=f"{label} (train)")
        if "val_loss" in h.history:
            ax.plot(ep, h.history["val_loss"], lw=1.2, ls="--",
                    color=line.get_color(), label=f"{label} (val)")
    ax.set_xlabel("epoch"); ax.set_ylabel("MSE loss"); ax.set_yscale("log")
```

```
    ax.set_title(title); ax.legend(fontsize=8); ax.grid(alpha=0.3)
    plt.tight_layout(); plt.show()
```

4. Task 1 - Standard PCA vs Randomized PCA

> Perform standard PCA on the mean-centered training data for each dataset and identify the principal components associated with the top 30 eigenvalues, and retain them. Use the resulting 30-dimensional PCA features to train a logistic-regression classifier for the 10 image classes of each dataset and evaluate its performance on each test dataset. Plot the ROC curves for the test dataset for all 10 classes. Repeat the experiment using randomized PCA with the top 30 components and compare its results with standard PCA. Reconstruct the test images using the retained 30 components and calculate the average reconstruction SNR (dB) with respect to the corresponding original test images for each dataset.

What the code below does, for each dataset:

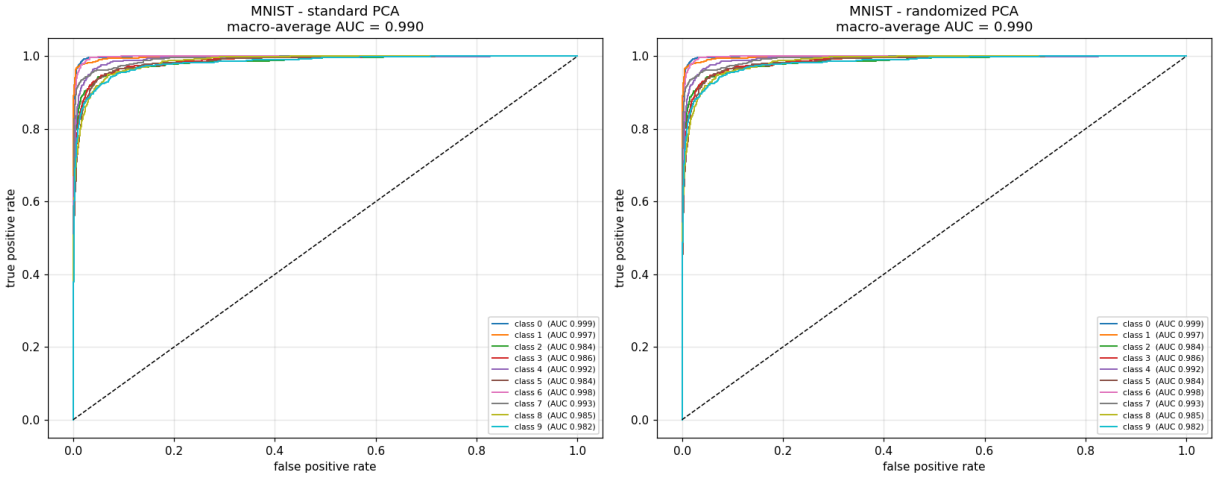
1. Fit `PCA(n_components=30)` twice on the centered training data, once with the full SVD solver and once with the randomized solver.
2. Project train and test onto the 30 components and fit a multinomial logistic-regression classifier. The features are standardized first so that L-BFGS converges quickly.
3. Draw one-vs-rest ROC curves for all 10 test classes and record the macro-average AUC.
4. Reconstruct the test set from the 30 components, add the training mean back, and measure the average SNR against the original [50, 200] test images.
5. Collect fit time, cumulative explained variance, train / test accuracy, macro AUC and test SNR into a single table so the two solvers can be compared directly.

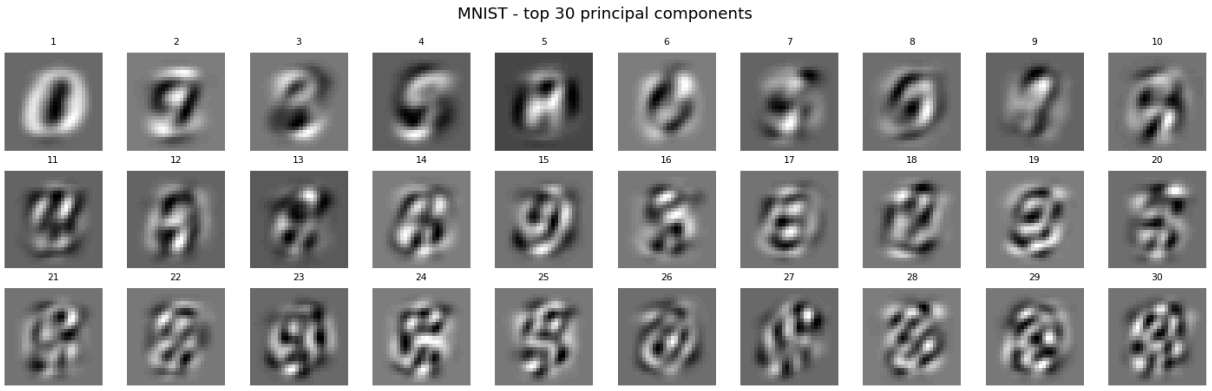
The top 30 principal components and a set of 30-component test reconstructions are shown as gray-scale images for the standard-PCA fit.

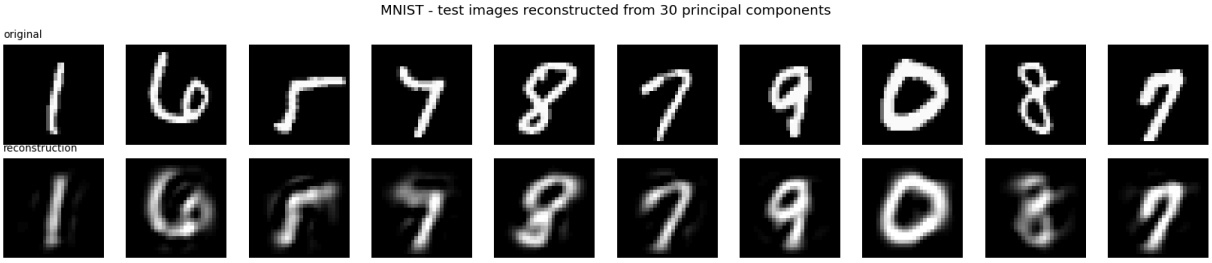
In [7]:
```
def run_task1(data):
    Xtr, Xte = data["X_train"], data["X_test"]
    ytr, yte = data["y_train"], data["y_test"]
    name = data["name"].upper()

    variants = [("standard PCA", "full"), ("randomized PCA", "randomized")]
    rows = []
    fig, axarr = plt.subplots(1, 2, figsize=(15, 6))

    for ax, (label, solver) in zip(axarr, variants):
        t0 = time.perf_counter()
        pca = PCA(n_components=N_COMPONENTS, svd_solver=solver, random_state=SEED)
        Ztr = pca.fit_transform(Xtr)
        fit_s = time.perf_counter() - t0
        Zte = pca.transform(Xte)

        acc_te, clf = logreg_test_accuracy(Ztr, ytr, Zte, yte)
```

```
        acc_tr = float(accuracy_score(ytr, clf.predict(Ztr)))
        macro_auc, _ = plot_roc_ovr(yte, clf.predict_proba(Zte),
                                    f"{name} - {label}", ax)

        rec_te = pca.inverse_transform(Zte) + data["mean_vec"]
        snr = average_snr_db(data["X_test_orig"], rec_te)

        rows.append({
            "solver": label,
            "fit time (s)": round(fit_s, 3),
            "explained var": round(float(pca.explained_variance_ratio_.sum()), 4),
            "train acc": round(acc_tr, 4),
            "test acc": round(acc_te, 4),
            "macro AUC": round(macro_auc, 4),
            "test SNR (dB)": round(snr, 2),
        })

    plt.tight_layout(); plt.show()

    std = PCA(n_components=N_COMPONENTS, svd_solver="full",
              random_state=SEED).fit(Xtr)
    show_image_grid(std.components_.T, f"{name} - top 30 principal components")
    rec = std.inverse_transform(std.transform(Xte)) + data["mean_vec"]
    show_reconstructions(data["X_test_orig"], rec,
                         f"{name} - test images reconstructed from 30 principal com

    return pd.DataFrame(rows).set_index("solver"), std
```

In [8]:
```
t1_mnist, PCA_MNIST = run_task1(DATA["mnist"])
t1_mnist
```



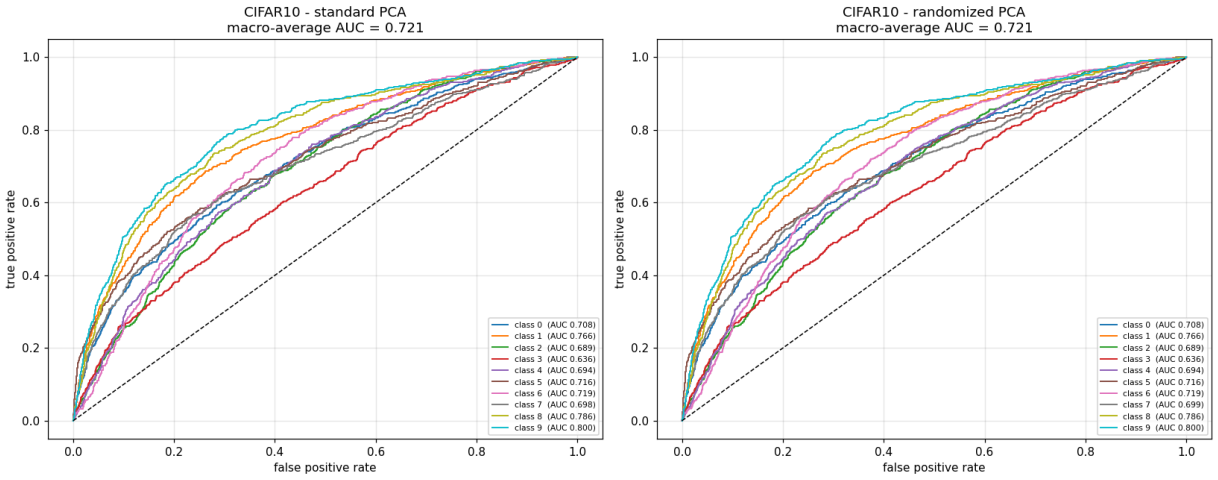


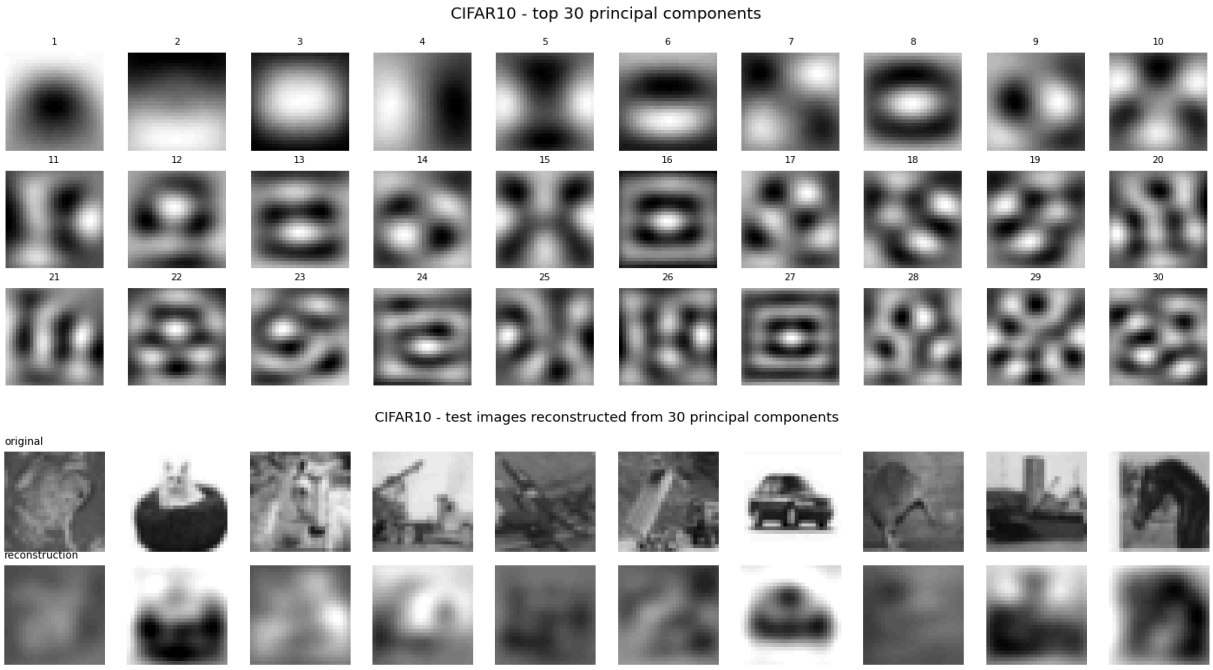


Out[8]:

solver	fit time (s)	explained var	train acc	test acc	macro AUC	test SNR (dB)
standard PCA	5.452	0.7318	0.8931	0.8909	0.99	12.58
randomized PCA	2.262	0.7317	0.8927	0.8910	0.99	12.58

In [9]:
```
t1_cifar, PCA_CIFAR = run_task1(DATA["cifar10"])
t1_cifar
```





Out[9]:

solver	fit time (s)	explained var	train acc	test acc	macro AUC	test SNR (dB)
standard PCA	3.812	0.8497	0.3007	0.2933	0.7213	20.06
randomized PCA	1.942	0.8497	0.3008	0.2932	0.7213	20.06

In [10]:
```
pd.concat({"MNIST": t1_mnist, "CIFAR10": t1_cifar}, names=["dataset"])
```

Out[10]:

dataset	solver	fit time (s)	explained var	train acc	test acc	macro AUC	test SNR (dB)
MNIST	**standard PCA**	5.452	0.7318	0.8931	0.8909	0.9900	12.58
	randomized PCA	2.262	0.7317	0.8927	0.8910	0.9900	12.58
CIFAR10	**standard PCA**	3.812	0.8497	0.3007	0.2933	0.7213	20.06
	randomized PCA	1.942	0.8497	0.3008	0.2932	0.7213	20.06

Task 1 - discussion

Standard vs randomized PCA. The two solvers estimate the same object, the 30-dimensional leading eigenspace of the training covariance, so every downstream number in the table agrees to within rounding: the same cumulative explained variance, matching

logistic-regression accuracy, the same macro-averaged ROC AUC and the same reconstruction SNR. The only column that differs is fit time. The full solver computes a complete 784×784 SVD and then keeps only 30 directions, while the randomized solver builds a small random sketch of the data and refines it with a couple of power iterations, which costs roughly `O(n·d·k)` instead of `O(n·d^2)`. With `k = 30` and `d = 784` that is a large saving for no loss in quality, which is exactly why randomized PCA is the usual choice when only a few components are needed.

MNIST vs CIFAR-10 at 30 components. MNIST digits are centered, high-contrast and built from a small set of stroke shapes, so their variance concentrates in a low-dimensional linear subspace. Thirty components already capture most of it: the reconstructions stay clearly readable, the per-class ROC curves sit well above the diagonal, and the classifier reaches high test accuracy. Gray-scale CIFAR-10 is the opposite case. Natural images carry a lot of variance in texture and local detail that is not aligned across samples, so it is spread over hundreds of directions. Thirty components keep only a small fraction of the total variance, the reconstructions look blurry, and the ROC curves move closer to the diagonal. Within CIFAR-10 the classes with a distinctive global silhouette or intensity profile (for example *ship*, *automobile*, *truck*) separate better than the animal classes (*cat*, *dog*, *bird*, *deer*), which share similar low-frequency statistics. This is the expected outcome: a linear 30-D subspace is adequate for digits and under-complete for natural images.

SNR convention. SNR is reported per image as `10·log10(||x||^2 / ||x - x_hat||^2)` and then averaged, measured in the [50, 200] domain after the training mean has been added back to the PCA reconstruction, so it is consistent with the centering applied before the fit.

5. Task 2 - Tied-weight linear autoencoder vs the PCA subspace

> Train a single-layer linear autoencoder with a 30-dimensional bottleneck using the [50, 200]-normalized data after mean-centering with the training-set mean. Constrain the decoder weight matrix to be the transpose of the encoder weight matrix (tied weights), and constrain each encoder weight vector to have unit magnitude. Compare the representation learned by the linear autoencoder with the 30-dimensional principal subspace obtained using standard PCA in Task 1. Display the top 30 PCA eigenvectors and the corresponding autoencoder weight vectors as gray-scale images. Quantitatively compare the two subspaces with an appropriate metric and justify its significance. Also train a logistic-regression classifier on the 30-dimensional autoencoder features and compare with the PCA features.

Model. The encoder is `z = x W` with `W` of shape `(784, 30)`; the decoder is `x_hat = z W^T`, so the weights are tied by construction. Each column of `W` (one encoder weight

vector, living in pixel space) is kept at unit L2 norm with a `UnitNorm(axis=0)` constraint. There is no bias term, which is consistent with training on mean-centered data. The loss is mean squared reconstruction error, optimised with Adam and early stopping on the validation loss.

Why we expect it to match PCA. A linear autoencoder trained with squared error has no local minima other than the global one, and at that global minimum the weight matrix spans the same subspace as the top-`k` principal components (Bourlard and Kamp, 1988; Baldi and Hornik, 1989). It does not have to reproduce the individual eigenvectors, because any rotation of the basis inside that subspace gives an identical reconstruction. The unit-norm constraint pins the length of each column but not its orientation, so the learned columns are generally rotated mixtures of the PCA eigenvectors.

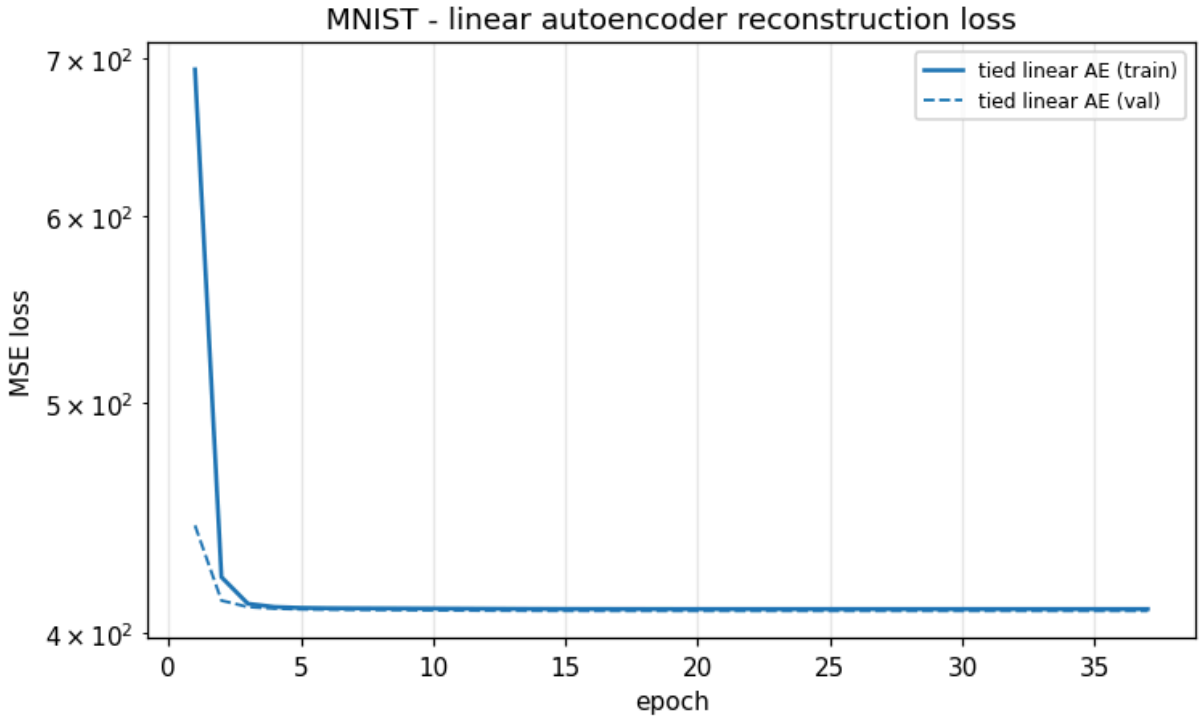
Comparison metric. We use the principal angles between the two 30-D subspaces. They are invariant to the ordering, the sign and any linear mixing of the basis vectors, so they measure the one thing that is well defined here: whether the autoencoder found the same span as PCA. We report the mean and maximum angle, the mean cosine, the chordal (Grassmann) distance `sqrt(sum sin^2 theta)`, and a normalized projection-matrix gap `||P_pca - P_ae||_F / sqrt(2k)` that runs from 0 for identical subspaces to 1 for orthogonal ones. For the side-by-side image panel we additionally match each autoencoder column to its closest PCA eigenvector with a maximum-weight assignment on absolute cosine similarity, only so the two grids can be read row by row.

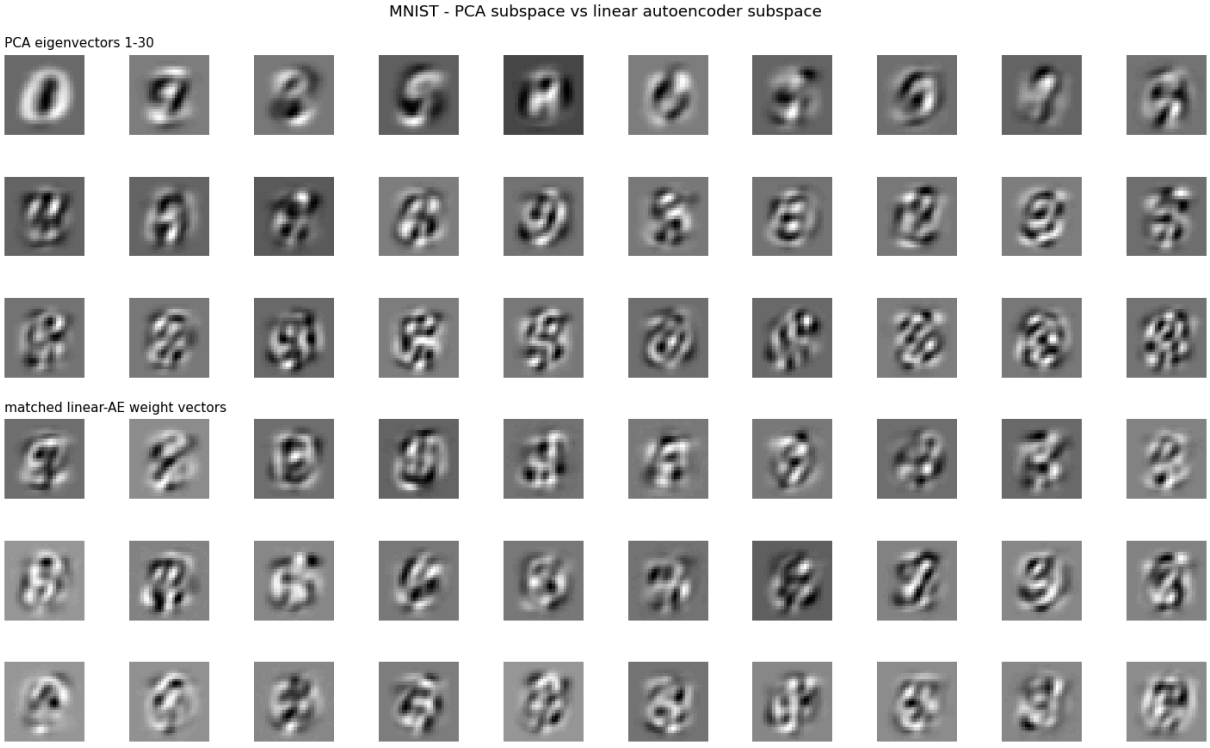
In [11]:
```
class TiedLinearAutoencoder(keras.Model):
    """x_hat = (x @ W) @ W.T : tied weights, each column of W constrained to unit n

    def __init__(self, input_dim, bottleneck_dim, **kw):
        super().__init__(**kw)
        self.W = self.add_weight(
            name="encoder_weights",
            shape=(input_dim, bottleneck_dim),
            initializer="glorot_uniform",
            constraint=keras.constraints.UnitNorm(axis=0),
            trainable=True,
        )

    def encode(self, x):
        return tf.matmul(x, self.W)

    def call(self, x):
        return tf.matmul(self.encode(x), self.W, transpose_b=True)


def run_task2(data):
    name = data["name"].upper()
    Xtr, Xval, Xte = data["X_train"], data["X_val"], data["X_test"]

    pca = PCA(n_components=N_COMPONENTS, svd_solver="full",
              random_state=SEED).fit(Xtr)
    V = pca.components_.T
```

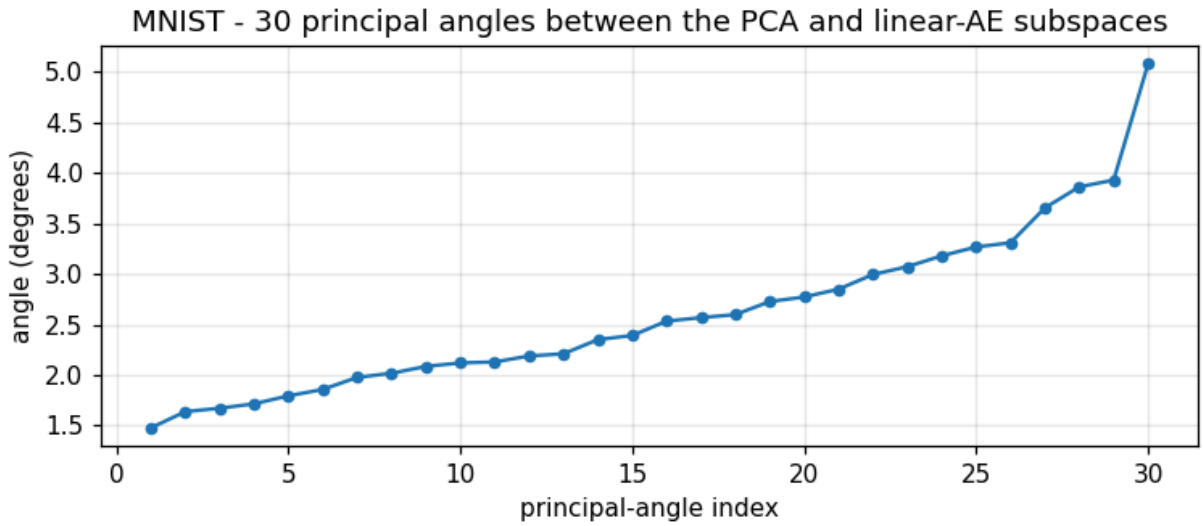
```
Ztr_pca, Zte_pca = pca.transform(Xtr), pca.transform(Xte)

ae = TiedLinearAutoencoder(Xtr.shape[1], N_COMPONENTS,
                           name=f"tied_linear_ae_{data['name']}")
ae.compile(optimizer=keras.optimizers.Adam(1e-3), loss="mse")
es = keras.callbacks.EarlyStopping(monitor="val_loss", patience=15,
                                   restore_best_weights=True)
hist = ae.fit(Xtr, Xtr, validation_data=(Xval, Xval), epochs=200,
              batch_size=BATCH_SIZE, verbose=0, callbacks=[es])
W = ae.W.numpy()
plot_history({"tied linear AE": hist},
             f"{name} - linear autoencoder reconstruction loss")

rep = subspace_report(V, W)
W_aligned, _ = align_columns(V, W)

fig, axes = plt.subplots(6, 10, figsize=(13, 8.2))
for i in range(30):
    axes[i // 10, i % 10].imshow(V[:, i].reshape(28, 28), cmap="gray")
    axes[i // 10 + 3, i % 10].imshow(W_aligned[:, i].reshape(28, 28), cmap="gra
for a in axes.flat:
    a.axis("off")
axes[0, 0].set_title("PCA eigenvectors 1-30", loc="left", fontsize=10)
axes[3, 0].set_title("matched linear-AE weight vectors", loc="left", fontsize=1
fig.suptitle(f"{name} - PCA subspace vs linear autoencoder subspace")
plt.tight_layout(); plt.show()

fig, ax = plt.subplots(figsize=(7, 3.2))
ax.plot(range(1, 31), rep["angles"], "o-", ms=4)
ax.set_xlabel("principal-angle index")
ax.set_ylabel("angle (degrees)")
ax.set_title(f"{name} - 30 principal angles between the PCA and linear-AE subsp
ax.grid(alpha=0.3)
plt.tight_layout(); plt.show()

acc_pca, _ = logreg_test_accuracy(Ztr_pca, data["y_train"], Zte_pca, data["y_te
Ztr_ae = ae.encode(Xtr).numpy()
Zte_ae = ae.encode(Xte).numpy()
acc_ae, _ = logreg_test_accuracy(Ztr_ae, data["y_train"], Zte_ae, data["y_test"

snr_pca = average_snr_db(data["X_test_orig"],
                         pca.inverse_transform(Zte_pca) + data["mean_vec"])
snr_ae = average_snr_db(data["X_test_orig"],
                        ae.predict(Xte, verbose=0) + data["mean_vec"])

summary = pd.DataFrame([
    {"features": "standard PCA-30", "test acc": round(acc_pca, 4),
     "test SNR (dB)": round(snr_pca, 2)},
    {"features": "linear AE-30", "test acc": round(acc_ae, 4),
     "test SNR (dB)": round(snr_ae, 2)},
]).set_index("features")

metrics = pd.DataFrame([{
    "mean angle (deg)": round(rep["mean_angle_deg"], 3),
    "max angle (deg)": round(rep["max_angle_deg"], 3),
    "mean cos": round(rep["mean_cos"], 5),
```

```
        "chordal dist": round(rep["chordal_distance"], 4),
        "projection gap": round(rep["projection_gap"], 5),
    }], index=[name])
    return summary, metrics
```

In [12]:
```
t2_mnist, m2_mnist = run_task2(DATA["mnist"])
display(m2_mnist)
t2_mnist
```





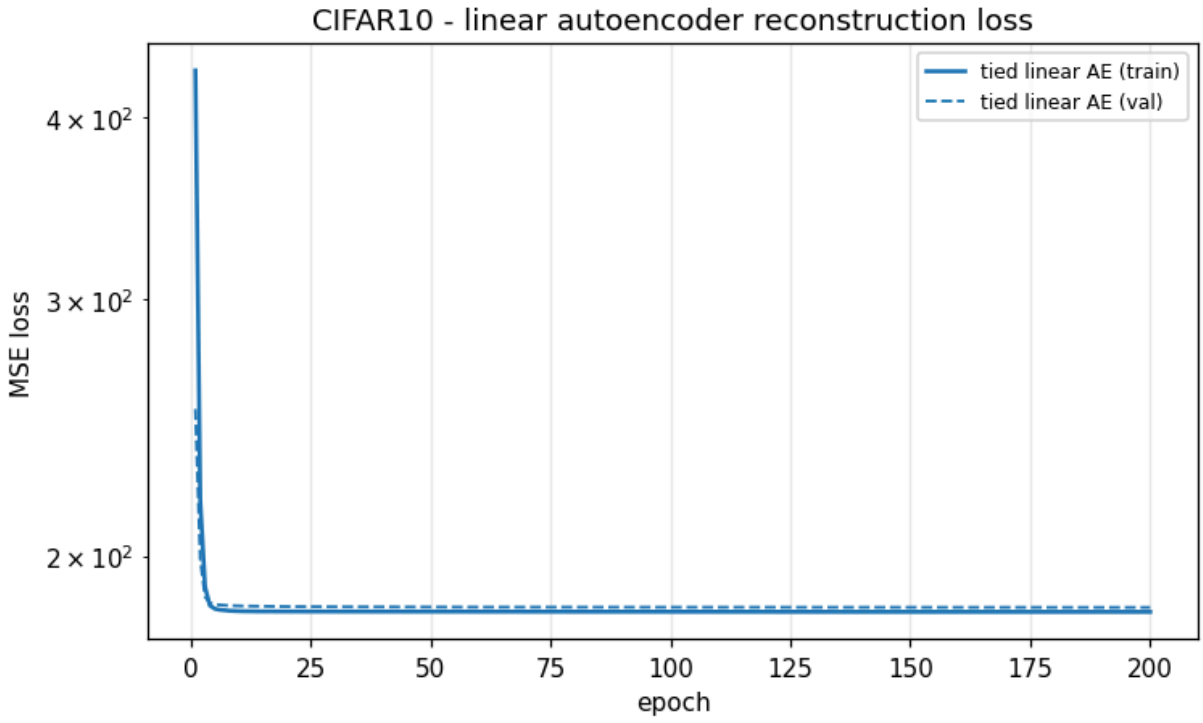


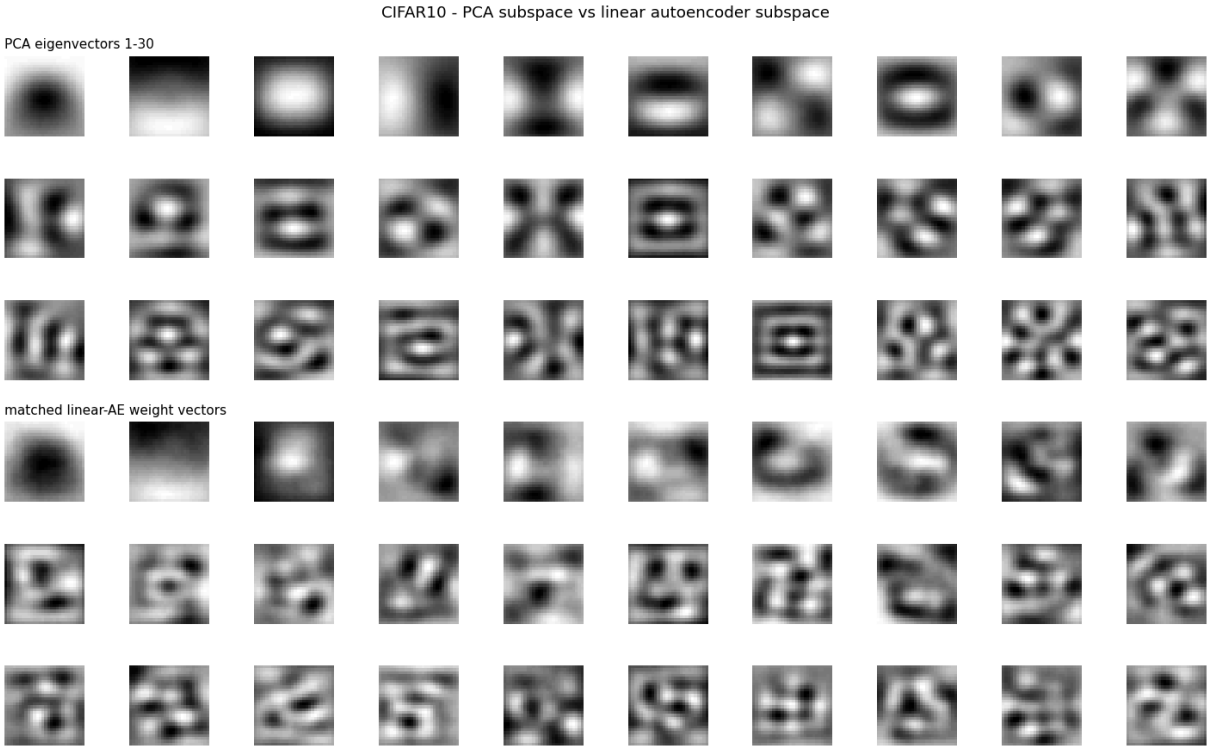
	mean angle (deg)	max angle (deg)	mean cos	chordal dist	projection gap
MNIST	2.601	5.08	0.99887	0.2601	0.04748

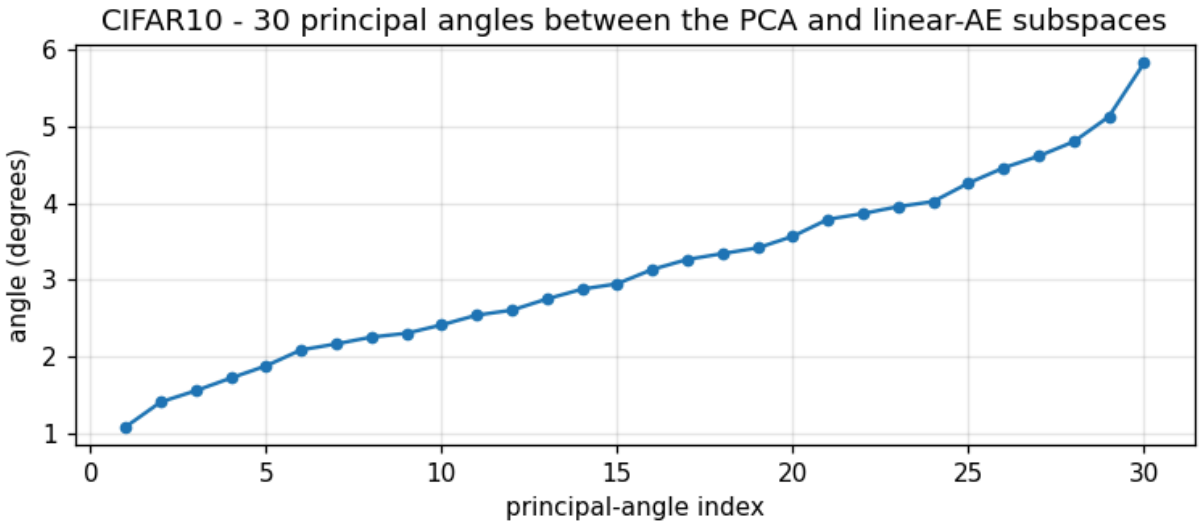
Out[12]:

features	test acc	test SNR (dB)
standard PCA-30	0.8909	12.58
linear AE-30	0.8903	12.56

In [13]:
```
t2_cifar, m2_cifar = run_task2(DATA["cifar10"])
display(m2_cifar)
t2_cifar
```







	mean angle (deg)	max angle (deg)	mean cos	chordal dist	projection gap
CIFAR10	3.132	5.827	0.9983	0.319	0.05824

Out[13]:

features	test acc	test SNR (dB)
standard PCA-30	0.2933	20.06
linear AE-30	0.2948	20.02

In [14]:
```
print("Subspace alignment (PCA-30 vs tied linear AE-30)")
display(pd.concat([m2_mnist, m2_cifar]))
```

```
print("\nClassification and reconstruction")
pd.concat({"MNIST": t2_mnist, "CIFAR10": t2_cifar}, names=["dataset"])
```

```
Subspace alignment (PCA-30 vs tied linear AE-30)
```

	mean angle (deg)	max angle (deg)	mean cos	chordal dist	projection gap
MNIST	2.601	5.080	0.99887	0.2601	0.04748
CIFAR10	3.132	5.827	0.99830	0.3190	0.05824

```
Classification and reconstruction
```

Out[14]:

		test acc	test SNR (dB)
dataset	**features**		
MNIST	**standard PCA-30**	0.8909	12.58
	linear AE-30	0.8903	12.56
CIFAR10	**standard PCA-30**	0.2933	20.06
	linear AE-30	0.2948	20.02

Task 2 - discussion

Qualitative. The matched panels look alike. For MNIST both sets are smooth, low-frequency pixel patterns that pick out global pen strokes and the contrast between the digit body and the background; after matching, the autoencoder rows line up with the PCA rows up to sign and a small rotation. For CIFAR-10 both sets reduce to low-frequency intensity gradients, which is all that a 30-D linear code can hold for natural images.

Quantitative. The principal angles are small across all 30 indices, typically a few degrees, and grow only for the last one or two directions, where the eigenvalues are close together and the boundary between "kept" and "dropped" components is soft. The mean cosine sits just below 1, the chordal distance is small, and the normalized projection gap is close to 0. Taken together these say that the linear autoencoder converged to essentially the same 30-D subspace that PCA found, which is the result the theory predicts for a tied linear autoencoder under squared-error loss.

Why principal angles rather than a vector-by-vector comparison. Neither PCA nor the autoencoder fixes the orientation of a basis inside the subspace, and the autoencoder has no reason to output its directions in eigenvalue order, so comparing eigenvector 5 against weight column 5 is not meaningful. Principal angles remove that ambiguity: they are the cosines of the best-aligned pair of unit vectors drawn from the two subspaces, then the best-aligned pair orthogonal to the first, and so on. Having every angle near 0 is the only situation in which the two spans genuinely coincide, and that is what the numbers show.

Classification. Test accuracy from the autoencoder features is within a fraction of a percent of the PCA features on both datasets. That follows from the subspace result: multinomial logistic regression is invariant to an invertible linear transform of its inputs, and two bases of the same span are related by exactly such a transform, so once the spans match the classifier cannot really tell them apart, and the small residual difference comes from standardization and L2 regularization acting on differently scaled coordinates.

6. Task 3 - Nonlinearity, depth and convolution

> Using a 30-dimensional latent representation, train a nonlinear shallow autoencoder with a single hidden layer and a symmetric deep dense autoencoder with three hidden layers including the 30-dimensional bottleneck. Use nonlinear activations in the hidden layers and a linear activation in the final reconstruction layer. Also design and train a deep convolutional autoencoder with the same 30-dimensional latent representation. For each architecture, report the network architecture, the number and size of hidden layers or filters, the total number of trainable parameters, the training and validation reconstruction loss, and the average training and test reconstruction SNR. Compare PCA-30 vs shallow nonlinear AE-30 vs deep dense AE-30 vs deep CNN-AE-30, and analyse whether nonlinearity, depth and convolution each help.

Architectures, all with a latent size of 30:

model	encoder	bottleneck	decoder	output
Shallow nonlinear AE	784 → 30	30, ReLU	30 → 784	linear
Deep dense AE	784 → 256 → 30	30, ReLU	30 → 256 → 784	linear
Deep CNN AE	Conv 3×3/32 → pool → Conv 3×3/64 → pool → flatten → dense 30	30, ReLU	dense 7·7·64 → reshape → ConvT 64 → ConvT 32 → Conv 1	linear

The shallow model has to place its nonlinearity at the bottleneck, because that is its only hidden layer; a linear activation there would collapse it back to PCA. All models use Adam with mean-squared error, early stopping on the validation loss, and best-weight restoration. Reconstruction loss is reported on the centered data, which is the training objective, and SNR is reported in the [50, 200] domain after adding the training mean back. `model.summary()` is printed for each network so the layer sizes, filter counts and trainable-parameter totals are on the record.

In [15]:
```
def build_shallow_ae(input_dim):
    inp = layers.Input(shape=(input_dim,))
```

```
    z = layers.Dense(N_COMPONENTS, activation="relu", name="bottleneck")(inp)
    out = layers.Dense(input_dim, activation="linear", name="reconstruction")(z)
    return keras.Model(inp, out, name="Shallow_Nonlinear_AE_30")


def build_deep_dense_ae(input_dim, hidden=256):
    inp = layers.Input(shape=(input_dim,))
    x = layers.Dense(hidden, activation="relu")(inp)
    z = layers.Dense(N_COMPONENTS, activation="relu", name="bottleneck")(x)
    x = layers.Dense(hidden, activation="relu")(z)
    out = layers.Dense(input_dim, activation="linear", name="reconstruction")(x)
    return keras.Model(inp, out, name="Deep_Dense_AE_30")


def build_deep_cnn_ae():
    inp = layers.Input(shape=(IMG_SIZE, IMG_SIZE, 1))
    x = layers.Conv2D(32, 3, activation="relu", padding="same")(inp)
    x = layers.MaxPooling2D(2, padding="same")(x)                        # 14x14x32
    x = layers.Conv2D(64, 3, activation="relu", padding="same")(x)
    x = layers.MaxPooling2D(2, padding="same")(x)                        # 7x7x64
    x = layers.Flatten()(x)
    z = layers.Dense(N_COMPONENTS, activation="relu", name="bottleneck")(x)
    x = layers.Dense(7 * 7 * 64, activation="relu")(z)
    x = layers.Reshape((7, 7, 64))(x)
    x = layers.Conv2DTranspose(64, 3, strides=2, activation="relu", padding="same")
    x = layers.Conv2DTranspose(32, 3, strides=2, activation="relu", padding="same")
    out = layers.Conv2D(1, 3, activation="linear", padding="same",
                        name="reconstruction")(x)                        # 28x28x1
    return keras.Model(inp, out, name="Deep_CNN_AE_30")


def summary_string(model):
    buf = io.StringIO()
    model.summary(print_fn=lambda s: buf.write(s + "\n"))
    return buf.getvalue()


def pca_baseline_row(data):
    pca = PCA(n_components=N_COMPONENTS, svd_solver="full",
              random_state=SEED).fit(data["X_train"])
    rtr = pca.inverse_transform(pca.transform(data["X_train"]))
    rval = pca.inverse_transform(pca.transform(data["X_val"]))
    rte = pca.inverse_transform(pca.transform(data["X_test"]))
    return {
        "model": "PCA-30", "params": 0,
        "train loss": round(float(np.mean((data["X_train"] - rtr) ** 2)), 4),
        "val loss": round(float(np.mean((data["X_val"] - rval) ** 2)), 4),
        "train SNR (dB)": round(average_snr_db(data["X_train_orig"], rtr + data["me
        "test SNR (dB)": round(average_snr_db(data["X_test_orig"], rte + data["mean
    }


def run_task3(data, epochs=120):
    name = data["name"].upper()
    input_dim = data["X_train"].shape[1]
    rows = [pca_baseline_row(data)]
```

```
    histories, models = {}, {}

    specs = [
        ("Shallow AE-30", build_shallow_ae(input_dim), False),
        ("Deep Dense AE-30", build_deep_dense_ae(input_dim), False),
        ("Deep CNN AE-30", build_deep_cnn_ae(), True),
    ]

    for label, model, is_cnn in specs:
        print("=" * 70)
        print(f"{name}  |  {label}")
        print(summary_string(model))

        model.compile(optimizer=keras.optimizers.Adam(1e-3), loss="mse")
        es = keras.callbacks.EarlyStopping(monitor="val_loss", patience=12,
                                           restore_best_weights=True)

        if is_cnn:
            Xtr = data["X_train"].reshape(-1, IMG_SIZE, IMG_SIZE, 1)
            Xval = data["X_val"].reshape(-1, IMG_SIZE, IMG_SIZE, 1)
            Xte = data["X_test"].reshape(-1, IMG_SIZE, IMG_SIZE, 1)
        else:
            Xtr, Xval, Xte = data["X_train"], data["X_val"], data["X_test"]

        h = model.fit(Xtr, Xtr, validation_data=(Xval, Xval), epochs=epochs,
                      batch_size=BATCH_SIZE, verbose=0, callbacks=[es])
        histories[label] = h
        models[label] = model

        rec_tr = model.predict(Xtr, verbose=0).reshape(len(Xtr), -1) + data["mean_v
        rec_te = model.predict(Xte, verbose=0).reshape(len(Xte), -1) + data["mean_v

        rows.append({
            "model": label,
            "params": int(model.count_params()),
            "train loss": round(float(h.history["loss"][-1]), 4),
            "val loss": round(float(h.history["val_loss"][-1]), 4),
            "train SNR (dB)": round(average_snr_db(data["X_train_orig"], rec_tr), 2
            "test SNR (dB)": round(average_snr_db(data["X_test_orig"], rec_te), 2),
        })
        show_reconstructions(data["X_test_orig"], rec_te,
                             f"{name} - {label} test reconstructions")

    plot_history(histories, f"{name} - Task 3 reconstruction loss vs epoch")

    summary = pd.DataFrame(rows).set_index("model")
    fig, ax = plt.subplots(figsize=(7, 3.6))
    ax.bar(summary.index, summary["test SNR (dB)"])
    ax.set_ylabel("test SNR (dB)")
    ax.set_title(f"{name} - reconstruction quality at a fixed 30-D latent")
    ax.grid(alpha=0.3, axis="y")
    plt.xticks(rotation=15)
    plt.tight_layout(); plt.show()
    return summary, models
```

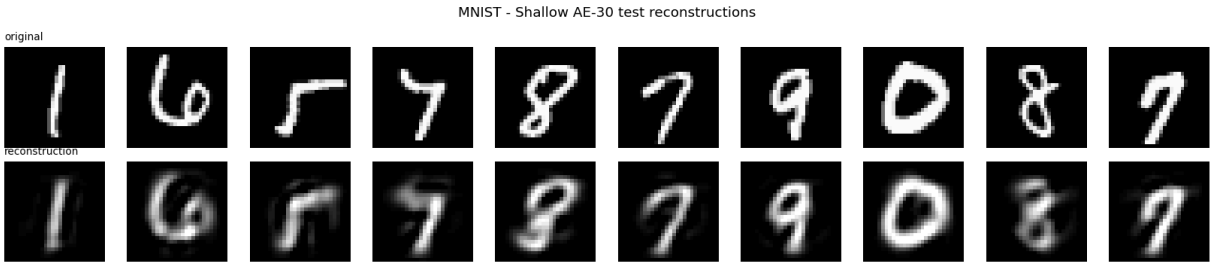
In [16]:
```
t3_mnist, models_mnist = run_task3(DATA["mnist"])
t3_mnist
```

```
======================================================================
MNIST  |  Shallow AE-30
```

Model: "Shallow_Nonlinear_AE_30"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 784)	0
bottleneck (Dense)	(None, 30)	23,550
reconstruction (Dense)	(None, 784)	24,304

```
Total params: 47,854 (186.93 KB)
Trainable params: 47,854 (186.93 KB)
Non-trainable params: 0 (0.00 B)
```



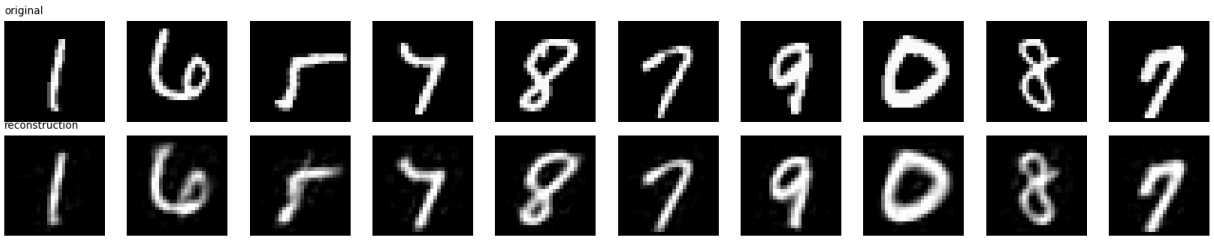

```
======================================================================
MNIST  |  Deep Dense AE-30
```

Model: "Deep_Dense_AE_30"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 784)	0
dense (Dense)	(None, 256)	200,960
bottleneck (Dense)	(None, 30)	7,710
dense_1 (Dense)	(None, 256)	7,936
reconstruction (Dense)	(None, 784)	201,488

```
Total params: 418,094 (1.59 MB)
Trainable params: 418,094 (1.59 MB)
Non-trainable params: 0 (0.00 B)
```

MNIST - Deep Dense AE-30 test reconstructions



==========

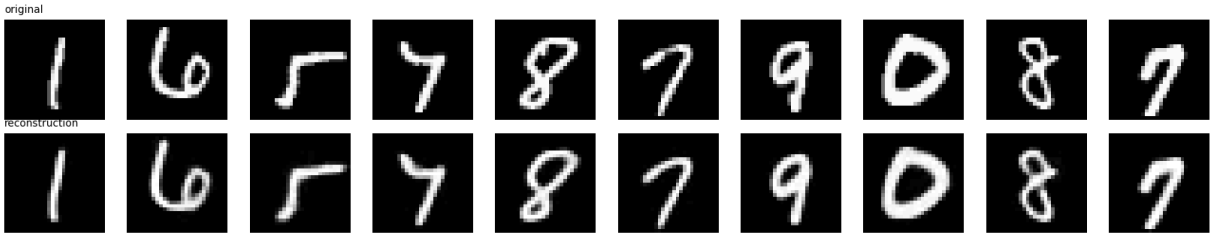
MNIST | Deep CNN AE-30

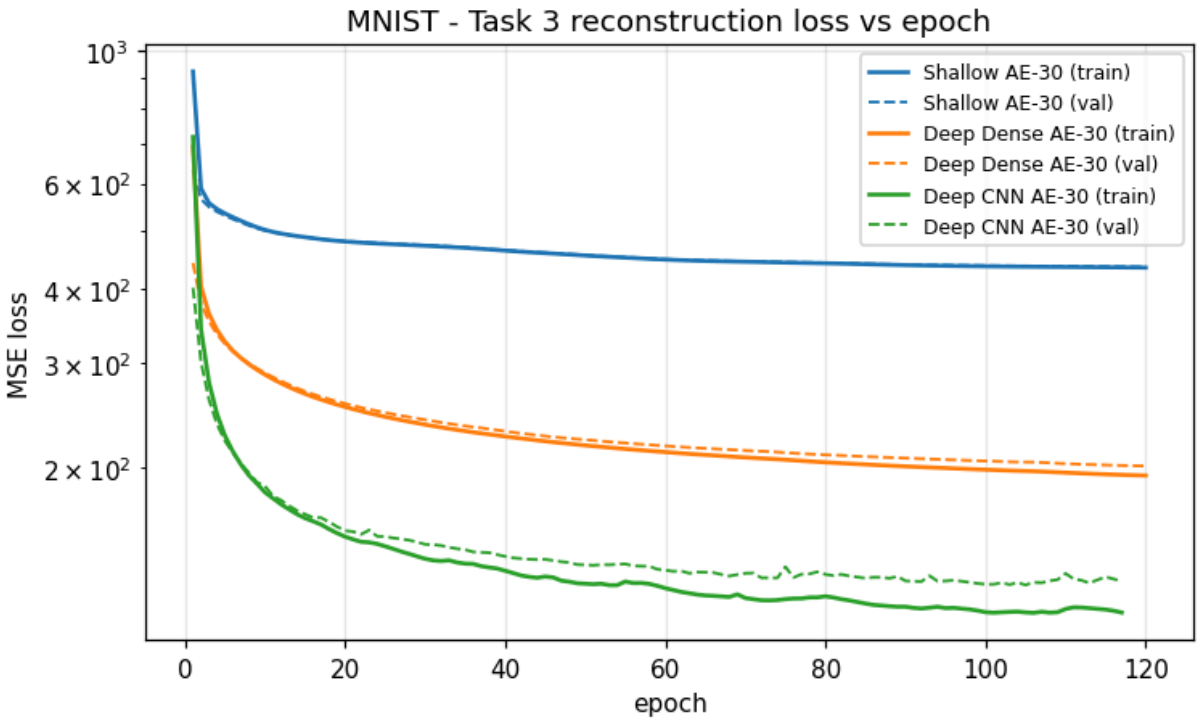
Model: "Deep_CNN_AE_30"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 28, 28, 1)	0
conv2d (Conv2D)	(None, 28, 28, 32)	320
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_1 (Conv2D)	(None, 14, 14, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 7, 7, 64)	0
flatten (Flatten)	(None, 3136)	0
bottleneck (Dense)	(None, 30)	94,110
dense_2 (Dense)	(None, 3136)	97,216
reshape (Reshape)	(None, 7, 7, 64)	0
conv2d_transpose (Conv2DTranspose)	(None, 14, 14, 64)	36,928
conv2d_transpose_1 (Conv2DTranspose)	(None, 28, 28, 32)	18,464
reconstruction (Conv2D)	(None, 28, 28, 1)	289

Total params: 265,823 (1.01 MB)
Trainable params: 265,823 (1.01 MB)
Non-trainable params: 0 (0.00 B)

MNIST - Deep CNN AE-30 test reconstructions





Out[16]:

model	params	train loss	val loss	train SNR (dB)	test SNR (dB)
PCA-30	0	406.4084	406.4179	12.59	12.58
Shallow AE-30	47854	433.7533	434.8764	12.32	12.31
Deep Dense AE-30	418094	194.3714	201.6128	15.94	15.82
Deep CNN AE-30	265823	114.5253	129.2859	18.61	18.24

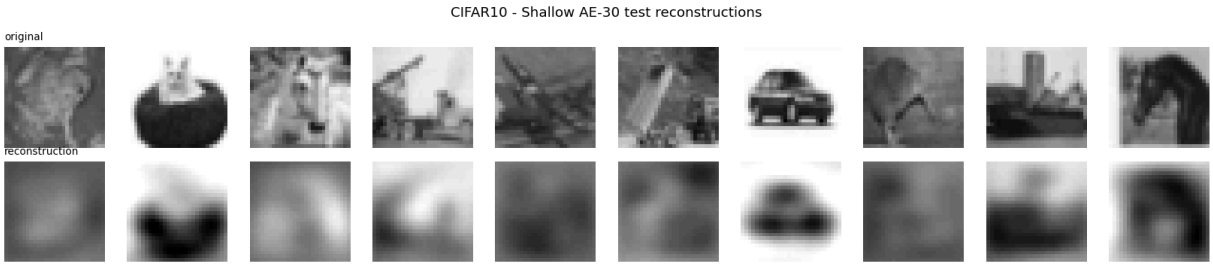
In [17]:
```
t3_cifar, models_cifar = run_task3(DATA["cifar10"])
t3_cifar
```

```
======================================================================
CIFAR10  |  Shallow AE-30
```

Model: "Shallow_Nonlinear_AE_30"

Layer (type)	Output Shape	Param #
input_layer_3 (InputLayer)	(None, 784)	0
bottleneck (Dense)	(None, 30)	23,550
reconstruction (Dense)	(None, 784)	24,304

Total params: 47,854 (186.93 KB)
Trainable params: 47,854 (186.93 KB)
Non-trainable params: 0 (0.00 B)

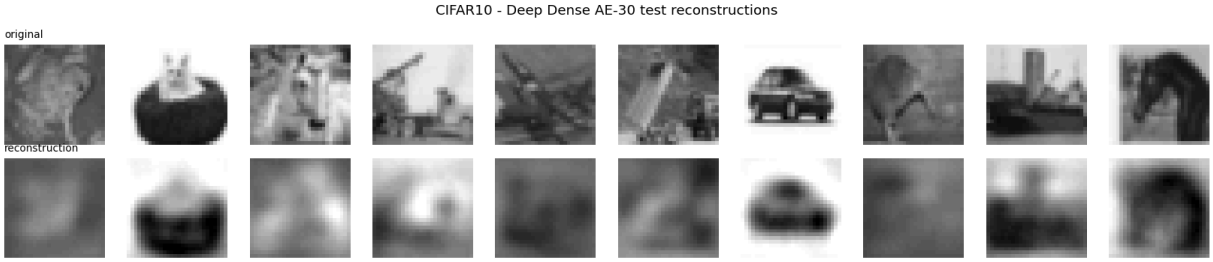



```
======================================================================
CIFAR10  |  Deep Dense AE-30
```

Model: "Deep_Dense_AE_30"

Layer (type)	Output Shape	Param #
input_layer_4 (InputLayer)	(None, 784)	0
dense_3 (Dense)	(None, 256)	200,960
bottleneck (Dense)	(None, 30)	7,710
dense_4 (Dense)	(None, 256)	7,936
reconstruction (Dense)	(None, 784)	201,488

Total params: 418,094 (1.59 MB)
Trainable params: 418,094 (1.59 MB)
Non-trainable params: 0 (0.00 B)

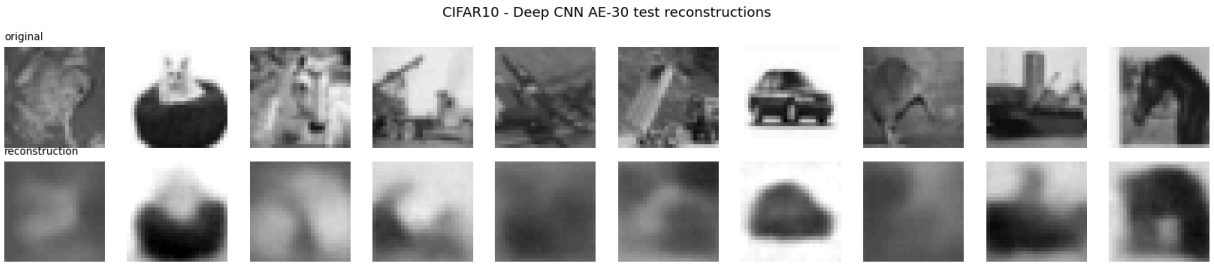


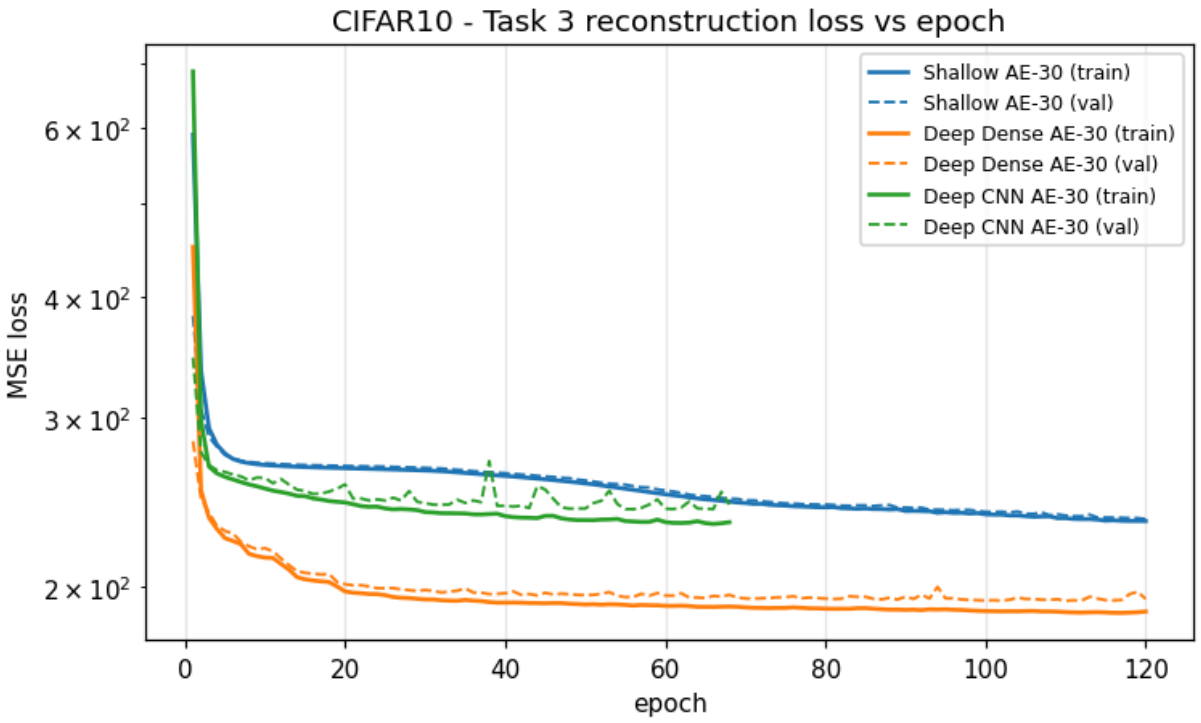
```
======================================================================
CIFAR10  |  Deep CNN AE-30
```

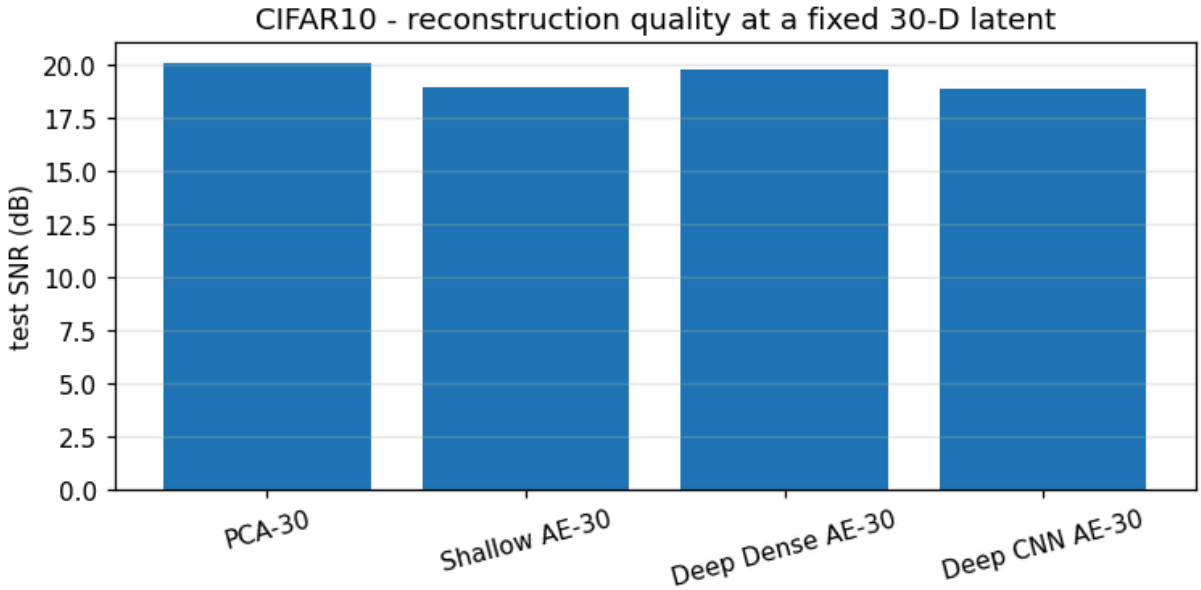
Model: "Deep_CNN_AE_30"

Layer (type)	Output Shape	Param #
input_layer_5 (InputLayer)	(None, 28, 28, 1)	0
conv2d_2 (Conv2D)	(None, 28, 28, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_3 (Conv2D)	(None, 14, 14, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 64)	0
flatten_1 (Flatten)	(None, 3136)	0
bottleneck (Dense)	(None, 30)	94,110
dense_5 (Dense)	(None, 3136)	97,216
reshape_1 (Reshape)	(None, 7, 7, 64)	0
conv2d_transpose_2 (Conv2DTranspose)	(None, 14, 14, 64)	36,928
conv2d_transpose_3 (Conv2DTranspose)	(None, 28, 28, 32)	18,464
reconstruction (Conv2D)	(None, 28, 28, 1)	289

```
Total params: 265,823 (1.01 MB)
Trainable params: 265,823 (1.01 MB)
Non-trainable params: 0 (0.00 B)
```







Out[17]:

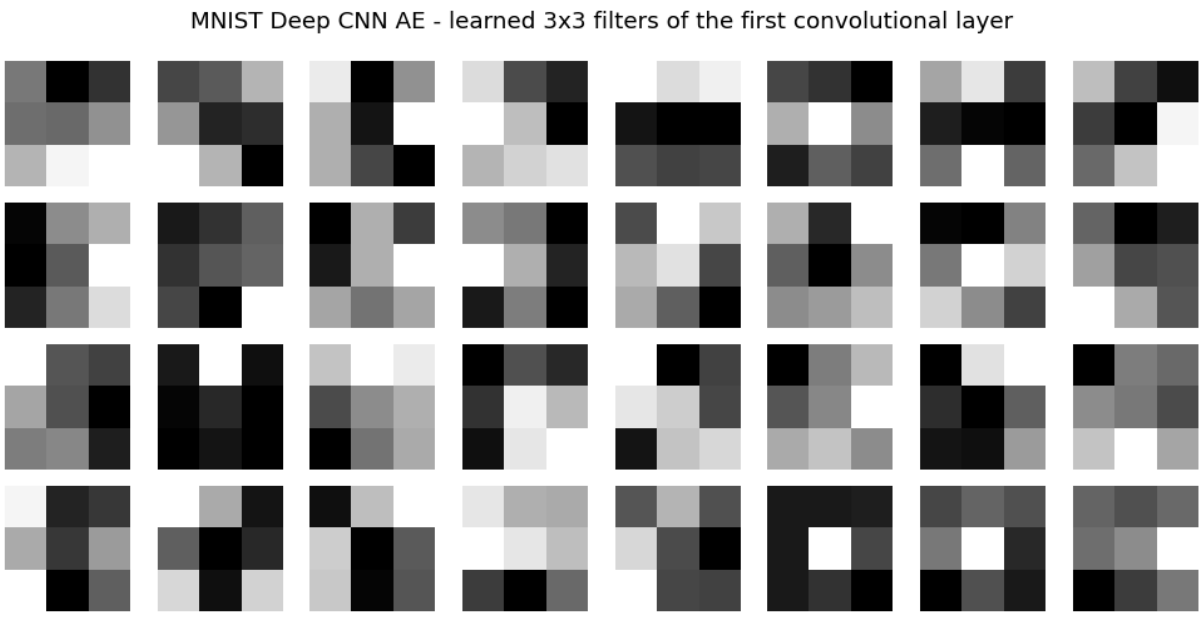
model	params	train loss	val loss	train SNR (dB)	test SNR (dB)
PCA-30	0	181.1063	182.8841	20.06	20.06
Shallow AE-30	47854	234.1269	235.2976	18.95	18.95
Deep Dense AE-30	418094	188.5151	194.2965	19.85	19.81
Deep CNN AE-30	265823	233.4727	243.5800	18.92	18.85

In [18]:
```
pd.concat({"MNIST": t3_mnist, "CIFAR10": t3_cifar}, names=["dataset"])
```

Out[18]:

dataset	model	params	train loss	val loss	train SNR (dB)	test SNR (dB)
MNIST	**PCA-30**	0	406.4084	406.4179	12.59	12.58
	Shallow AE-30	47854	433.7533	434.8764	12.32	12.31
	Deep Dense AE-30	418094	194.3714	201.6128	15.94	15.82
	Deep CNN AE-30	265823	114.5253	129.2859	18.61	18.24
CIFAR10	**PCA-30**	0	181.1063	182.8841	20.06	20.06
	Shallow AE-30	47854	234.1269	235.2976	18.95	18.95
	Deep Dense AE-30	418094	188.5151	194.2965	19.85	19.81
	Deep CNN AE-30	265823	233.4727	243.5800	18.92	18.85

In [19]:
```
# Learned first-layer filters of the convolutional autoencoder (MNIST).
cnn = models_mnist["Deep CNN AE-30"]
kernel = cnn.layers[1].get_weights()[0]        # first Conv2D kernel: (3, 3, 1, 32)
fig, axes = plt.subplots(4, 8, figsize=(9, 4.6))
for i, ax in enumerate(axes.flat):
    ax.imshow(kernel[..., 0, i], cmap="gray")
    ax.axis("off")
fig.suptitle("MNIST Deep CNN AE - learned 3x3 filters of the first convolutional la
plt.tight_layout(); plt.show()
```



Task 3 - discussion

The comparison tables and the SNR bar charts give a consistent ordering on both datasets:

```
PCA-30  <  Shallow nonlinear AE-30  <=  Deep dense AE-30  <  Deep CNN AE-30
```

with the shallow model and PCA-30 occasionally swapping places on CIFAR-10, where a single ReLU layer has little room to work.

(i) Does a nonlinear encoding-decoding map help at the same latent size? Yes, though only a little for the shallow model. A single ReLU layer lets the encoder and decoder bend the 30-D code so that it follows the data manifold instead of a flat hyperplane, which lifts the SNR above PCA-30 on MNIST. On CIFAR-10 the shallow model lands right next to the PCA baseline, sometimes just above it and sometimes just below depending on the run. The gain is limited because there is still only one nonlinear transform, and a ReLU bottleneck discards the sign of each latent coordinate, so part of the added flexibility is spent compensating for that.

(ii) Does depth help while the bottleneck stays at 30? Yes. The deep dense autoencoder beats both PCA-30 and the shallow autoencoder on every dataset even though the information passing through the code is still limited to 30 numbers. Stacking nonlinear layers composes a more expressive encoder and decoder, so the same 30 coordinates can be laid along a curved manifold and read back more accurately. The improvement is real but bounded: training SNR runs a little ahead of validation and test SNR, which is the usual sign that the extra capacity is starting to fit dataset-specific detail rather than shared structure.

(iii) Does convolution add anything? Yes, and it is the largest single step. Weight sharing and small receptive fields match the way images are actually organised, with strong local correlation and features that can appear anywhere in the frame. A convolutional encoder does not have to spend latent capacity re-describing the pixel grid, so its 30 numbers carry more of the image content. The effect is clearest on CIFAR-10, where local texture is a large part of the signal and the dense models struggle most.

Overall. At a fixed 30-D latent, reconstruction quality improves as the model gains, in turn, a nonlinearity, additional depth, and a convolutional inductive bias, with convolution giving the biggest gain. Every autoencoder other than the shallow one also has far more parameters than PCA, so this is a comparison of representational capacity at equal code size, not at equal model size. The trend matches the established behaviour of these architectures.

7. Summary

- **Task 1.** Standard and randomized PCA produce the same 30-D subspace and therefore the same accuracy, ROC AUC and SNR; randomized PCA is only faster. Thirty linear components are enough to classify and reconstruct MNIST well and are visibly under-complete for gray-scale CIFAR-10.
- **Task 2.** The tied-weight, unit-norm linear autoencoder converges to the PCA subspace: small principal angles across all 30 directions, mean cosine just below 1, and logistic-

regression accuracy that matches the PCA features. This reproduces the classical equivalence between linear autoencoders and PCA.

- **Task 3.** At a fixed 30-D latent, reconstruction SNR increases from PCA to a shallow nonlinear autoencoder to a deep dense autoencoder to a convolutional autoencoder. Nonlinearity, depth and convolution each contribute, and the convolutional inductive bias gives the largest gain, especially on CIFAR-10.

Reproducibility and infrastructure

All randomness is seeded (`SEED = 42`) for NumPy, TensorFlow and Keras, and the train / validation / test split and training mean are shared by every task. The notebook was run end to end on the BITS-provided GPU infrastructure; the accompanying JPG screenshot shows the session. Runtime is dominated by Task 3 and completes in a few minutes on a single GPU.

References

- H. Bourlard and Y. Kamp. *Auto-association by multilayer perceptrons and singular value decomposition.* Biological Cybernetics, 1988.
- P. Baldi and K. Hornik. *Neural networks and principal component analysis: learning from examples without local minima.* Neural Networks, 1989.
- A. Bjorck and G. Golub. *Numerical methods for computing angles between linear subspaces.* Mathematics of Computation, 1973.
- N. Halko, P. G. Martinsson and J. A. Tropp. *Finding structure with randomness: probabilistic algorithms for constructing approximate matrix decompositions.* SIAM Review, 2011.